PAYTM BACKEND INTERVIEW GUIDE

Java · Spring Boot · Kafka · System Design

Senior Engineer (7.5 yrs) | 20 Deep Q&As | Syntax-Highlighted Code | 2025

20	7	STAR	■

Code Color Legend

Keyword Type/Class String Comment Method Number Annotation

■ SECTION 1 · CORE JAVA

Q1. Explain JVM memory model — Heap, Stack, Metaspace and how GC works.

■ Answer & Explanation

The JVM divides memory into distinct regions — crucial for tuning Paytm's payment services:

- **Heap:** All objects and class instances. Young Gen (Eden+S0+S1) + Old Gen (Tenured). GC happens here.
- **Stack:** Per-thread. Method frames, local primitives, references. Auto-freed on method return. Never GC'd.
- **Metaspace (Java 8+):** Class metadata. Grows dynamically from native memory. Replaced PermGen.
- **PC Register:** Per-thread pointer to current JVM instruction.

GC Algorithm	Pause Time	Best For	JVM Flag
Serial GC	High (full STW)	Single-core, small heaps	-XX:+UseSerialGC
G1GC (default Java 9+)	100–200ms	Balanced — general use	-XX:+UseG1GC
ZGC (Java 11+)	< 1ms	Low-latency payment APIs	-XX:+UseZGC
Shenandoah	< 10ms	Large heaps, concurrent	-XX:+UseShenandoahGC

■ ***Tip:*** *For Paytm payment API (SLA < 200ms): use ZGC. Tune: -Xmx4g -Xms4g -XX:+UseZGC -XX:MaxGCPauseMillis=5*

Q2. What is the Java Memory Model (JMM)? Explain volatile and happens-before.

■ Answer & Explanation

The JMM defines rules for thread visibility. Without it, each CPU core's cache means one thread's writes may never be visible to another.

◆ Java

```
// ■ BROKEN — without volatile, JIT may cache 'instance' in CPU register
private static Singleton instance;

// ■ CORRECT — volatile: guarantees visibility + prevents instruction reordering
private static volatile Singleton instance;

public static Singleton getInstance() {
if (instance == null) { // first check (no lock — fast path)
synchronized (Singleton.class) { // only lock when null
if (instance == null) { // second check (inside lock)
instance = new Singleton(); // safe — volatile stops partial construct
}
}
}
return instance;
}

// Without volatile: CPU reorders steps — another thread sees non-null but
// reads an incompletely constructed object (fields not yet initialized!)
```

■■ Critical: Self-invocation this.method() skips the Spring proxy — @Transactional and @Async will be SILENTLY IGNORED!

■ ***Tip:*** *happens-before rules: unlock → lock; volatile write → read; thread.start() → run(); run() → join() caller.*

Q3. CompletableFuture — how does it work and how do you use it at Paytm scale?

■ Answer & Explanation

CompletableFuture enables non-blocking async pipelines. Parallelizing sequential downstream calls at Paytm dramatically cuts latency.

◆ Java

```java
// ■ SEQUENTIAL — 300ms + 300ms + 100ms = 700ms total
User user = fetchUser(id); // 300ms — blocks thread
Wallet wallet = fetchWallet(id); // 300ms — blocks thread
KYC kyc = fetchKYC(id); // 100ms — blocks thread

// ■ PARALLEL — max(300, 300, 100) = 300ms total (60% faster!)
// Always provide a CUSTOM executor — never use ForkJoinPool.commonPool() in prod!
ExecutorService executor = Executors.newFixedThreadPool(20);

CompletableFuture<User> userFuture = CompletableFuture.supplyAsync(() -> fetchUser(id), executor);
CompletableFuture<Wallet> walletFuture = CompletableFuture.supplyAsync(() -> fetchWallet(id), executor);
CompletableFuture<KYC> kycFuture = CompletableFuture.supplyAsync(() -> fetchKYC(id), executor);

// Wait for ALL three to complete
CompletableFuture.allOf(userFuture, walletFuture, kycFuture).thenRun(() -> {
User user = userFuture.join(); // non-blocking — already done
Wallet wallet = walletFuture.join();
KYC kyc = kycFuture.join();
buildResponse(user, wallet, kyc);
}).exceptionally(ex -> {
log.error("Checkout fetch failed", ex);
return null;
});

// Key API Reference:
// supplyAsync(Supplier, Executor) — run async task on executor
// thenApply(Function) — transform result (same thread)
// thenApplyAsync(Function, Executor) — transform on separate thread
// thenCompose(Function<T,CF<U>>) — flatMap: chain dependent future
// exceptionally(Function<Throwable>) — fallback on any exception
// allOf(CF...) — complete when ALL complete
// anyOf(CF...) — complete when FIRST completes
```

■ ***Tip:*** *Real Paytm impact: checkout API 1,200ms → 140ms by parallelizing user-profile, wallet-balance, and fraud-score via CompletableFuture.allOf().*

Q4. ReentrantLock vs synchronized — differences and when to use which?

■ Answer & Explanation

Feature	synchronized	ReentrantLock
Try locking without blocking	■ Blocks forever	■ tryLock(timeout, unit)
Interruptible wait	■ No	■ lockInterruptibly()
Fairness (FIFO ordering)	■ No	■ new ReentrantLock(true)
Multiple condition vars	■ One wait/notify set	■ Unlimited Condition objects
Read-write split	■ No	■ ReentrantReadWriteLock
Code complexity	■ Implicit block-scoped	■■ Explicit try/finally required

Java

```java
// ReentrantLock — ALWAYS unlock in finally!
ReentrantLock lock = new ReentrantLock();

public void processPayment(Payment payment) {
try {
// tryLock with timeout — avoids deadlock risk
if (!lock.tryLock(500, TimeUnit.MILLISECONDS)) {
throw new TimeoutException("Could not acquire payment lock");
}
try {
debitWallet(payment); // critical section
callBankGateway(payment);
} finally {
lock.unlock(); // ALWAYS release — even on exception
}
} catch (InterruptedException e) {
Thread.currentThread().interrupt(); // restore interrupted flag!
}
}

// Multiple Conditions — fine-grained signaling (Producer-Consumer)
ReentrantLock lock = new ReentrantLock();
Condition notFull = lock.newCondition(); // producer waits here when queue full
Condition notEmpty = lock.newCondition(); // consumer waits here when queue empty
// Signal only producers OR only consumers — more efficient than notifyAll()
```

■ ***Tip:*** *Use synchronized for simple cases. Use ReentrantLock for: tryLock timeout, fair ordering, or multiple conditions (e.g., bounded blocking queue).*

■ SECTION 2 · SPRING BOOT (Advanced)

Q5. How does @Transactional work internally? Explain propagation + isolation levels.

■ Answer & Explanation

Spring wraps @Transactional methods with a proxy (CGLIB for classes, JDK proxy for interfaces). The proxy intercepts the call, starts a tx, runs your method, then commits or rolls back.

```
◆ Java
```

```java
// ■■ THE MOST COMMON @Transactional MISTAKE: Self-Invocation
// this.method() bypasses the Spring proxy → @Transactional SILENTLY ignored!

@Service
public class OrderService {

// ■ WRONG — processPayment() runs WITHOUT any transaction
public void createOrder(Order order) {
this.processPayment(order); // direct call, skips Spring proxy
}

@Transactional
public void processPayment(Order order) { /* runs without tx context! */ }

// ■ FIX 1 — inject self (Spring returns the proxy, not 'this')
@Autowired
private OrderService self;

public void createOrderFixed(Order order) {
self.processPayment(order); // goes through proxy ✓
}
// ■ FIX 2 (cleaner) — extract processPayment() to a SEPARATE @Service bean
}

// PROPAGATION EXAMPLES:
@Transactional(propagation = Propagation.REQUIRES_NEW) // new tx, suspend outer
public void saveAuditLog(AuditEntry entry) {
auditRepo.save(entry); // saved even if the outer tx rolls back ✓
}

@Transactional(propagation = Propagation.NESTED) // savepoint in outer tx
public void processLineItem(LineItem item) {
lineItemRepo.save(item); // rollback here = partial undo, outer tx survives
}

@Transactional(isolation = Isolation.SERIALIZABLE) // highest isolation
public void debitWallet(String userId, BigDecimal amount) {
// No phantom reads, no dirty reads, no non-repeatable reads
// Full serialization — use for critical payment debits
}

@Transactional(rollbackFor = Exception.class, // rollback on checked exceptions
noRollbackFor = InsufficientFundsException.class) // NOT on this one
public void charge(Payment payment) throws InsufficientFundsException {
// InsufficientFunds = expected business outcome, should NOT roll back tx
}
```

Isolation Level	Dirty Read	Non-Repeatable	Phantom	Paytm Use
READ_UNCOMMITTED	■ Yes	■ Yes	■ Yes	■ Never
READ_COMMITTED	■ No	■ Yes	■ Yes	■ General

REPEATABLE_READ	■ No	■ No	■ Yes	■ Balance reads
SERIALIZABLE	■ No	■ No	■ No	■ Payment debit

■ ***Tip:*** *For wallet debit: SERIALIZABLE or optimistic locking (@Version). Prevents double-spend when two threads debit same wallet simultaneously.*

Q6. How does Spring AOP work? Write a custom logging + idempotency aspect.

■ Answer & Explanation

AOP separates cross-cutting concerns from business logic. Spring creates proxy beans at runtime — this is exactly how @Transactional, @Cacheable, and @Async work automatically.

◆ Java

```java
// ■■ Step 1: Custom Annotation ■■
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
public @interface Idempotent {
long ttlHours() default 24;
}

// ■■ Step 2: Aspect — intercepts methods at runtime via proxy ■■
@Aspect
@Component
public class PaymentAspect {

// ① Timing + Logging — wraps ALL payment service methods
@Around("execution(* com.paytm.payment.service.*.*(..))")
public Object logAndTime(ProceedingJoinPoint pjp) throws Throwable {
String method = pjp.getSignature().toShortString();
long start = System.currentTimeMillis();
try {
Object result = pjp.proceed(); // invoke the REAL method
log.info("[OK] {} in {}ms", method, System.currentTimeMillis()-start);
return result;
} catch (Exception e) {
log.error("[ERR] {} → {}", method, e.getMessage());
metrics.counter("payment.errors","method",method).increment();
throw e; // ALWAYS re-throw!
}
}

// ② Idempotency — intercepts @Idempotent annotated methods
@Around("@annotation(idempotent)")
public Object enforceIdempotency(ProceedingJoinPoint pjp,
Idempotent idempotent) throws Throwable {
String key = getCurrentRequest().getHeader("Idempotency-Key");
String redisKey = "idem:" + key;
String cached = redis.get(redisKey);

if (cached != null) { // already processed — return cached
return objectMapper.readValue(cached, getReturnType(pjp));
}
Object result = pjp.proceed();
redis.setex(redisKey, idempotent.ttlHours()*3600, serialize(result));
return result;
}
}

// ■■ Step 3: Usage — clean, declarative ■■
@PostMapping("/payment")
@Idempotent(ttlHours = 24) // runs at most once per idempotency key
public PaymentResponse pay(@RequestBody PaymentRequest req) {
return paymentService.process(req);
}
```

■ ***Tip:*** *CGLIB proxy (default in Spring Boot) requires a no-arg constructor. JDK proxy (for interfaces) is slightly faster. Mention both — shows depth of Spring internals knowledge.*

Q7. Explain Circuit Breaker with Resilience4j — states, config, fallback pattern.

■ Answer & Explanation

State	Behavior	Transition Trigger
CLOSED	Normal — all calls pass through	→ OPEN when failure rate ≥ threshold
OPEN	Fail-fast — no actual calls made, fallback triggered	→ HALF_OPEN after waitDurationInOpenState
HALF_OPEN	N probe calls allowed to test service recovery	→ CLOSED on success; → OPEN on failure

◆ application.yml

```
# application.yml — Resilience4j config for payment gateway
resilience4j:
circuitbreaker:
instances:
paymentGateway:
slidingWindowSize: 10 # evaluate last 10 calls
failureRateThreshold: 50 # OPEN if 50%+ fail
slowCallRateThreshold: 80 # OPEN if 80%+ are slow
slowCallDurationThreshold: 2000ms # 'slow' = > 2 seconds
waitDurationInOpenState: 10s # stay OPEN for 10s
permittedNumberOfCallsInHalfOpenState: 3 # 3 probes in HALF_OPEN
retry:
instances:
paymentGateway:
maxAttempts: 3
waitDuration: 1s
enableExponentialBackoff: true
exponentialBackoffMultiplier: 2 # delays: 1s → 2s → 4s
```

◆ Java

```java
@Service
public class PaymentGatewayService {

// Circuit Breaker + Retry + TimeLimiter — layered resilience
@CircuitBreaker(name = "paymentGateway", fallbackMethod = "paymentFallback")
@Retry(name = "paymentGateway")
@TimeLimiter(name = "paymentGateway")
public CompletableFuture<PaymentResponse> chargeAsync(PaymentRequest req) {
return CompletableFuture.supplyAsync(() -> gatewayClient.charge(req));
}

// Fallback — triggered when CB is OPEN or all retries exhausted
public CompletableFuture<PaymentResponse> paymentFallback(
PaymentRequest req, Exception ex) {
log.warn("[CB-FALLBACK] Gateway unavailable: {}", ex.getClass().getSimpleName());
kafkaProducer.send("payment-retry-queue", req.getUserId(), req); // queue retry
return CompletableFuture.completedFuture(
PaymentResponse.pending(req.getOrderId())); // return PENDING to client
}
}
```

■ ***Tip:*** *Stack all three: CB (opens at 50% failure) + Retry (3x, exponential backoff) + TimeLimiter (2s). Production resilience pattern at Paytm for all bank gateway calls.*

■ SECTION 3 · APACHE KAFKA (Highest Weight at Paytm)

Q8. Explain Kafka architecture — broker, partition, consumer group, ISR, offset.

■ Answer & Explanation

Concept	Description	Paytm Config
Topic	Immutable append-only log per domain event	One topic per event type: payment-events, wallet-events
Partition	Topic split into N ordered shards. Unit of parallelism.	Key=userId → same partition → ordered per user
Broker	Kafka server. Holds subset of partitions.	3–5 brokers in prod; replication factor=3
Leader	Handles all reads/writes for a partition.	1 leader per partition; auto-elected on failure
ISR	In-Sync Replicas — replicas caught up with leader.	acks=all → message committed only when all ISR ack
Consumer Group	Consumers sharing work. 1 partition → 1 consumer max.	group.id='payment-processor'; max parallel = partitions
Offset	Message position in partition. Consumer tracks here.	enable.auto.commit=false → manual commit

◆ Java

```java
// ■■ PRODUCER — reliable, idempotent, ordered per user ■■
Properties props = new Properties();
props.put("bootstrap.servers", "kafka1:9092,kafka2:9092,kafka3:9092");
props.put("acks", "all"); // wait for ALL ISR to ack
props.put("enable.idempotence", true); // broker deduplicates retries
props.put("retries", Integer.MAX_VALUE); // retry indefinitely
props.put("max.in.flight.requests.per.connection", 5);// safe with idempotence=true

// Key = userId → all events for same user → same partition → ordered ✓
ProducerRecord<String, PaymentEvent> record =
new ProducerRecord<>("payment-events", userId, paymentEvent);

producer.send(record, (metadata, exception) -> {
if (exception != null) log.error("Publish failed partition={}", metadata.partition());
});

// ■■ CONSUMER — manual commit, cooperative rebalancing ■■
props.put("group.id", "payment-processor");
props.put("auto.offset.reset", "earliest"); // don't miss messages on new group
props.put("enable.auto.commit", false); // manual commit = we control it
props.put("isolation.level", "read_committed"); // skip aborted tx messages
props.put("partition.assignment.strategy",
"org.apache.kafka.clients.consumer.CooperativeStickyAssignor"); // zero-downtime rebalance
props.put("max.poll.records", 100);
props.put("max.poll.interval.ms", 300000); // 5min — for heavy DB processing
```

■ ***Tip:*** *KEY: 10 partitions = max 10 parallel consumers. 11th consumer sits idle. Size partitions for peak throughput BEFORE launch — repartitioning live topics is painful and requires downtime.*

Q9. How do you achieve exactly-once semantics in Kafka for payment events?

■ Answer & Explanation

Semantic	Risk	Config	Paytm?
At-most-once	May LOSE messages	retries=0, auto.commit=true	■ Never
At-least-once	Duplicate messages possible	retries=N, manual commit	■ With idempotency key
Exactly-once (EOS)	Processed exactly once end-to-end	enable.idempotence + transactions	■ Kafka EOS or Redis key

◆ Java

```java
// ■■ OPTION A: Kafka Transactions (true EOS) ■■
producerProps.put("enable.idempotence", true);
producerProps.put("transactional.id", "payment-producer-pod-1"); // unique per JVM!
producerProps.put("acks", "all");

producer.initTransactions();
try {
producer.beginTransaction();
// Both sends are ATOMIC — either both commit or both abort
producer.send(new ProducerRecord<>("payment-events", key, paymentJson));
producer.send(new ProducerRecord<>("audit-log", key, auditJson));
producer.commitTransaction(); // ■ both visible atomically
} catch (ProducerFencedException e) {
producer.close(); // another instance took over — must close
} catch (Exception e) {
producer.abortTransaction(); // ■ both messages invisible to consumers
}

// Consumer: MUST use read_committed to skip uncommitted messages
consumerProps.put("isolation.level", "read_committed");


// ■■ OPTION B: At-least-once + Redis Idempotency Key (PAYTM'S ACTUAL APPROACH) ■■
// Simpler and more resilient than Kafka transactions in practice
@KafkaListener(topics = "payment-events", groupId = "payment-processor")
public void process(ConsumerRecord<String, PaymentEvent> record, Acknowledgment ack) {
String key = "processed:" + record.value().getIdempotencyKey();

if (Boolean.TRUE.equals(redis.hasKey(key))) {
log.info("[SKIP] Already processed: {}", key);
ack.acknowledge(); // commit offset — we've seen this before
return;
}

paymentService.process(record.value());
redis.opsForValue().set(key, "1", 24, TimeUnit.HOURS); // mark done
ack.acknowledge(); // commit offset AFTER successful processing
}
```

■ ***Tip:*** *Paytm REAL choice: at-least-once + Redis idempotency key. Kafka transactions add broker overhead and operational complexity. The Redis approach handles crashes, retries, and Kafka redeliveries elegantly.*

Q10. Kafka Consumer Rebalancing — what triggers it and how to minimize impact?

■ Answer & Explanation

Rebalancing: ALL consumers in a group STOP processing while partitions are redistributed. For real-time payment processing this causes dangerous processing gaps.

◆ Java

```java
// ■■ REBALANCE TUNING — minimize impact for payment consumers ■■

// CooperativeSticky = incremental rebalance — only move partitions that NEED moving
// Standard range/roundrobin: ALL consumers stop → ALL partitions reassigned (Stop-The-World)
// CooperativeSticky: ONLY affected partitions move → others continue processing ✓
props.put("partition.assignment.strategy",
"org.apache.kafka.clients.consumer.CooperativeStickyAssignor");

// Static membership — each pod gets a fixed ID
// On restart: Kafka waits for SAME consumer to rejoin instead of triggering rebalance
props.put("group.instance.id", "payment-consumer-pod-" + System.getenv("POD_INDEX"));

// Heartbeat / timeout tuning — prevent false rebalances from slow processing
props.put("heartbeat.interval.ms", 3000); // send heartbeat every 3s
props.put("session.timeout.ms", 45000); // consumer 'dead' if silent for 45s
props.put("max.poll.interval.ms", 300000); // 5min max between polls (for heavy DB work)
props.put("max.poll.records", 100); // don't over-fetch → slow processing → rebalance

// ■■ REBALANCE LISTENER — graceful commit before rebalance ■■
public class PaymentConsumer implements ConsumerAwareRebalanceListener {

@Override
public void onPartitionsRevokedBeforeCommit(Consumer<?,?> consumer,
Collection<TopicPartition> partitions) {
// Commit current offsets BEFORE partitions are taken away
consumer.commitSync(currentOffsets);
log.info("Revoked — committed offsets for {} partitions", partitions.size());
}

@Override
public void onPartitionsAssigned(Consumer<?,?> consumer,
Collection<TopicPartition> partitions) {
log.info("Assigned {} new partitions — warming caches", partitions.size());
}
}
```

■ ***Tip:*** *CooperativeSticky + static membership prevents 30–60 second processing gaps during rolling deployments. Critical for Paytm's zero-downtime payment processing.*

Q11. Design a Dead Letter Queue (DLQ) pattern in Kafka for failed payment events.

■ Answer & Explanation

DLQ routes repeatedly-failing messages to a separate topic instead of blocking the main pipeline. Paytm uses a multi-level retry topology:

◆ Java

```java
// ■■ MULTI-LEVEL RETRY TOPOLOGY ■■
// payment-events → (fail x1) → payment-retry-30s (retry after 30 seconds)
// → (fail x2) → payment-retry-5min (retry after 5 minutes)
// → (fail x3) → payment-DLQ (manual ops review + alert)

@KafkaListener(topics = "payment-events", groupId = "payment-processor")
public void processPayment(ConsumerRecord<String, PaymentEvent> record,
Acknowledgment ack) {
try {
paymentService.process(record.value());
ack.acknowledge(); // ■ success — commit offset

} catch (RetryableException e) {
// Transient (timeout, network) — send to retry topic with delay header
int retryCount = parseRetryCount(record.headers());
if (retryCount < 3) {
sendToRetryTopic(record, retryCount + 1);
} else {
sendToDLQ(record, e); // exhausted retries → DLQ
}
ack.acknowledge(); // commit — we re-queued it

} catch (NonRetryableException e) {
// Permanent error (invalid account, fraud block) → DLQ immediately
sendToDLQ(record, e);
ack.acknowledge();
}
}

// ■■ DLQ ENRICHMENT — ops team needs these headers for fast triage ■■
private void sendToDLQ(ConsumerRecord<String, PaymentEvent> record, Exception ex) {
RecordHeaders headers = new RecordHeaders(record.headers());
headers.add("X-Exception-Class", ex.getClass().getName().getBytes());
headers.add("X-Exception-Message", ex.getMessage().getBytes());
headers.add("X-Original-Topic", record.topic().getBytes());
headers.add("X-Original-Partition", String.valueOf(record.partition()).getBytes());
headers.add("X-Original-Offset", String.valueOf(record.offset()).getBytes());
headers.add("X-Failed-At", Instant.now().toString().getBytes());

dlqProducer.send(new ProducerRecord<>(
"payment-events-DLQ", null, record.key(), record.value(), headers));
log.error("[DLQ] key={} reason={}", record.key(), ex.getMessage());
}

@Scheduled(cron = "0 */30 * * * *") // replay DLQ every 30 minutes
public void reprocessDLQ() {
// Read DLQ, re-publish to main topic after root cause fixed
// Alert on-call (PagerDuty) if DLQ size > 100 messages
}
```

■ ***Tip:*** *Always enrich DLQ with exception, timestamp, original-topic, retry-count headers. Paytm ops team uses this for fast triage. Set up PagerDuty alert when DLQ depth exceeds threshold.*

■■ SECTION 4 · SYSTEM DESIGN

Q12. Design a Wallet System — concurrent debit/credit with strong consistency.

■ Answer & Explanation

◆ Java

```java
// ■■ ENTITY — @Version enables optimistic locking ■■
@Entity
@Table(name = "wallet_accounts",
indexes = @Index(columnList = "user_id", unique = true))
public class Wallet {
@Id @GeneratedValue
private Long id;

@Column(nullable = false, unique = true)
private String userId;

@Column(nullable = false, precision = 15, scale = 2)
private BigDecimal balance;

@Version // auto-incremented on every save
private int version; // concurrent update throws OptimisticLockException
}

// ■■ SERVICE — idempotency + optimistic lock + outbox ■■
@Service
@Transactional(isolation = Isolation.REPEATABLE_READ)
public class WalletService {

public void debit(String userId, BigDecimal amount, String idempotencyKey) {

// ① Idempotency check — prevent double-processing same request
if (txnRepo.existsByIdempotencyKey(idempotencyKey)) {
log.info("[SKIP] Already processed: {}", idempotencyKey);
return;
}

// ② Fetch wallet — @Version enables optimistic locking
Wallet wallet = walletRepo.findByUserId(userId)
.orElseThrow(() -> new WalletNotFoundException(userId));

// ③ Business validation
if (wallet.getBalance().compareTo(amount) < 0) {
throw new InsufficientFundsException(
"Balance " + wallet.getBalance() + " < Required " + amount);
}

// ④ Debit — save throws OptimisticLockException on concurrent update
wallet.setBalance(wallet.getBalance().subtract(amount));
walletRepo.save(wallet); // increments version; concurrent update fails here ✓

// ⑤ Record transaction for audit trail
txnRepo.save(WalletTransaction.debit(wallet.getId(), amount, idempotencyKey));

// ⑥ Outbox pattern — same DB tx = guaranteed event delivery to Kafka
outboxRepo.save(OutboxEvent.walletDebit(userId, amount, idempotencyKey));
// A separate poller reads outbox and publishes to Kafka
// Prevents dual-write: no risk of DB commit + Kafka publish both needed atomically
}
}
```

■ ***Tip:*** *OUTBOX PATTERN is critical: write transaction + event to DB in SAME transaction. Separate poller publishes to Kafka. Eliminates dual-write problem where DB succeeds but Kafka publish fails.*

Q13. Design a distributed Rate Limiter for Paytm's Payment API (Token Bucket algorithm).

■ Answer & Explanation

Algorithm	Behavior	Best For
Token Bucket	Tokens refill at fixed rate. Burst up to capacity allowed.	API rate limiting ■
Leaky Bucket	Process at constant rate. Bursts queued and smoothed.	Outgoing call shaping
Fixed Window	Simple counter per window (e.g., per minute).	Coarse limiting only
Sliding Window	Track exact request timestamps. Most accurate.	Audit / compliance

◆ Java

```
// ■■ REDIS LUA SCRIPT — atomic token bucket, no race conditions ■■
// Lua executes atomically in Redis — safe across distributed nodes!
private static final String TOKEN_BUCKET_SCRIPT = """
local key = KEYS[1]
local capacity = tonumber(ARGV[1])
local ttl = tonumber(ARGV[2])
local tokens = tonumber(redis.call('GET', key))
if tokens == nil then tokens = capacity end -- first request, full bucket
if tokens > 0 then
redis.call('SET', key, tokens - 1, 'EX', ttl)
return 1 -- ALLOWED
else
return 0 -- RATE LIMITED
end""";

@Component
public class RedisRateLimiter {
// 100 requests/minute per user, burst up to 200
public boolean isAllowed(String userId) {
Long result = (Long) redisTemplate.execute(
new DefaultRedisScript<>(TOKEN_BUCKET_SCRIPT, Long.class),
List.of("rate:" + userId),
"100", // ARGV[1] — max tokens (burst capacity)
"60" // ARGV[2] — TTL seconds (refill window)
);
return result != null && result == 1L;
}
}

// ■■ SPRING FILTER — applied at API Gateway, before business logic ■■
@Component @Order(1)
public class RateLimitFilter extends OncePerRequestFilter {
@Override
protected void doFilterInternal(HttpServletRequest req,
HttpServletResponse res,
FilterChain chain) throws IOException, ServletException {
String userId = jwtService.extractUserId(req);
if (!rateLimiter.isAllowed(userId)) {
res.setStatus(429);
res.setHeader("Retry-After", "60");
res.getWriter().write("{\"error\":\"Rate limit exceeded\"}");
return;
}
chain.doFilter(req, res);
}
}
```

■ ***Tip:*** *Lua script is atomic — no Redis WATCH/MULTI needed. Paytm applies: 100 req/min per user, 1000 req/min per merchant. Redis handles 100K rate-limit checks/sec easily.*

■ SECTION 5 · BEHAVIORAL (STAR Pattern)

STAR = **S**ituation · **T**ask · **A**ction · **R**esult — include specific numbers, timelines, outcomes.

Q14. Tell me about a system you designed from scratch. What challenges did you face?

■ Answer & Explanation

S	SITUATION	Payment reconciliation was fully manual — finance team running SQL queries at midnight, taking 4 hours. This caused delayed settlements and customer complaints about failed refunds.
T	TASK	Design and build an automated reconciliation system: 1M+ transactions/day, sub-1-hour SLA. Sole architect leading a team of 3.
A	ACTION	Event-driven: payment events → Kafka → reconciliation service → compared with bank file feed → discrepancies flagged → automated retry for minor diffs → ops alert for manual review. Used Spring Batch for large files, Redis for deduplication, Kafka exactly-once semantics. Delivered in 6 two-week sprints.
R	RESULT	Reconciliation time: 4 hours → 18 minutes. 95% automated. Finance team saved 120 man-hours/month. Zero settlement delays post-launch. Recognized in quarterly leadership review.

Q15. Describe a production incident you resolved. Your role and impact?

■ Answer & Explanation

S	SITUATION	3 AM alert. Payment service: 200ms → 8 second latency spike. 15,000 users affected. Revenue impact ~■50 lakh/minute.
T	TASK	On-call senior engineer — identify root cause and restore service within 30-minute SLA.
A	ACTION	Dashboards: HikariCP connection pool exhausted (max=10, pending=340). Root cause: Kafka consumer processing 10K records/batch, each making individual DB call, holding connections 90 seconds. Immediate fix: pool size 10→50, restart (5 min). Permanent fix next day: batch DB calls with IN clause, added HikariCP wait-time alert.
R	RESULT	Service restored in 8 minutes. As bonus, latency improved 200ms → 85ms. Post-mortem written. Monitoring added. Nominated for engineering recognition.

Q16. How did you significantly improve system performance? Specific example.

■ Answer & Explanation

S	SITUATION	Checkout API averaging 1,200ms. Business target: < 200ms for conversion improvement. Peak: 50,000 RPM.
T	TASK	Profile and optimize full checkout flow to < 200ms without changing the external API contract.

A	ACTION	async-profiler: (1) 3 sequential DB queries → 1 JOIN query. (2) Product catalog DB every request → Redis cache, 5-min TTL, 98% hit rate. (3) Fraud score was sync blocking → pre-computed async score refreshed every 30s. (4) CompletableFuture.allOf() to parallelize remaining I/O.
R	RESULT	Checkout: 1,200ms → 140ms. p99: 4s → 380ms. Conversion +3.2% = ■2Cr additional monthly revenue. Presented at engineering all-hands as case study.

Q17. Tell me about a conflict with a tech lead. How did you handle it?

■ Answer & Explanation

S	SITUATION	Tech lead wanted monolithic DB for new payment module for 'simplicity'. I believed this would cause scaling issues within 6 months given projected 10x user growth.
T	TASK	Convince team to adopt better architecture without causing conflict or damaging the relationship.
A	ACTION	Didn't argue in meeting. Prepared comparison: benchmarks, failure modes, migration cost at 10x scale. Had a private 1:1 with tech lead — acknowledged simplicity benefit, walked through risk data. Proposed middle path: single DB now but strict schema-per-domain separation for easy future extraction. Volunteered to write the migration runbook.
R	RESULT	Team agreed to middle path. 8 months later, domain extraction took 2 weeks instead of estimated 3 months. Relationship with tech lead improved. Included in promotion cycle.

■ SECTION 6 · CODING QUESTIONS (Senior Level)

Q18. Implement a Thread-Safe LRU Cache — O(1) get and put.

■ Answer & Explanation

◆ Java

```java
// LinkedHashMap(accessOrder=true) + ReentrantReadWriteLock
public class LRUCache<K, V> {

private final int capacity;
private final ReentrantReadWriteLock rwLock = new ReentrantReadWriteLock();
private final LinkedHashMap<K, V> cache;

public LRUCache(int capacity) {
this.capacity = capacity;
// accessOrder=true: get() moves the accessed entry to END (most recently used)
// removeEldestEntry: evicts the FIRST entry (least recently used) when over capacity
this.cache = new LinkedHashMap<>(capacity, 0.75f, true) {
@Override
protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {
return size() > LRUCache.this.capacity; // evict LRU when full
}
};
}

public V get(K key) {
// ■■ MUST use writeLock — accessOrder=true means get() modifies internal order!
rwLock.writeLock().lock();
try {
return cache.getOrDefault(key, null);
} finally {
rwLock.writeLock().unlock(); // always unlock in finally
}
}

public void put(K key, V value) {
rwLock.writeLock().lock();
try {
cache.put(key, value); // auto-evicts LRU entry if at capacity
} finally {
rwLock.writeLock().unlock();
}
}

public int size() {
rwLock.readLock().lock();
try { return cache.size(); }
finally { rwLock.readLock().unlock(); }
}
}

// Time: O(1) get and put | Space: O(capacity)
LRUCache<String, UserProfile> cache = new LRUCache<>(10_000);
cache.put("user123", profile);
UserProfile p = cache.get("user123"); // p is null if evicted or missing
```

■■ Critical: LinkedHashMap with accessOrder=true is NOT thread-safe for reads — get() modifies the node order. Must use writeLock for BOTH get and put. This is a famous interview gotcha!

Q19. Longest Substring Without Repeating Characters — O(n) sliding window.

■ Answer & Explanation

◆ Java

```java
// SLIDING WINDOW — two pointers + HashMap
// Window [left, right]: expand right, shrink left on duplicate character

public int lengthOfLongestSubstring(String s) {
Map<Character, Integer> lastSeen = new HashMap<>(); // char → last seen index
int maxLen = 0;
int left = 0; // left boundary of window

for (int right = 0; right < s.length(); right++) {
char c = s.charAt(right);

if (lastSeen.containsKey(c)) {
// Shrink left past the duplicate — but never move left backwards!
left = Math.max(left, lastSeen.get(c) + 1);
}

lastSeen.put(c, right); // update last seen position
maxLen = Math.max(maxLen, right - left + 1); // current window size
}
return maxLen;
}

// TRACE — "abcabcbb":
// right=0 'a': left=0 window="a" size=1 maxLen=1
// right=1 'b': left=0 window="ab" size=2 maxLen=2
// right=2 'c': left=0 window="abc" size=3 maxLen=3
// right=3 'a': left=1 window="bca" size=3 maxLen=3 ← skip past 'a' at index 0
// right=4 'b': left=2 window="cab" size=3 maxLen=3 ← skip past 'b' at index 1
// right=5 'c': left=3 window="abc" size=3 maxLen=3
// right=6 'b': left=5 window="cb" size=2 maxLen=3
// right=7 'b': left=7 window="b" size=1 maxLen=3
// RESULT: 3

// Time: O(n) | Space: O(min(m,n)) where m = charset size
```

Q20. Implement Thread-Safe Token Bucket Rate Limiter — in-process (no Redis).

■ Answer & Explanation

◆ Java

```java
// In-process Token Bucket — single JVM, no external dependency
public class TokenBucketRateLimiter {

private final long capacity; // max tokens (= max burst)
private final long refillTokens; // tokens added per interval
private final long refillIntervalMs; // refill interval (milliseconds)
private final AtomicLong tokens; // current tokens — thread-safe counter
private volatile long lastRefillTime; // volatile — visible to all threads

public TokenBucketRateLimiter(long capacity, long refillTokens, long intervalMs) {
this.capacity = capacity;
this.refillTokens = refillTokens;
this.refillIntervalMs = intervalMs;
this.tokens = new AtomicLong(capacity); // start with full bucket
this.lastRefillTime = System.currentTimeMillis();
}

// synchronized: make refill + check + decrement atomic
public synchronized boolean tryAcquire() {
refill(); // top up tokens based on elapsed time
if (tokens.get() > 0) {
tokens.decrementAndGet();
return true; // ■ allowed
}
return false; // ■ rate limited
}

private void refill() {
long now = System.currentTimeMillis();
long elapsed = now - lastRefillTime;
if (elapsed >= refillIntervalMs) {
long intervals = elapsed / refillIntervalMs;
long toAdd = intervals * refillTokens;
// Cap at capacity — don't exceed burst limit
tokens.set(Math.min(capacity, tokens.get() + toAdd));
lastRefillTime = now;
}
}
}

// Usage: 100 requests/second, burst allowed up to 200
TokenBucketRateLimiter limiter = new TokenBucketRateLimiter(
200, // capacity — burst up to 200 requests
100, // refill 100 tokens
1000 // every 1,000ms = 1 second
);

if (!limiter.tryAcquire()) {
throw new RateLimitExceededException("Too many requests. Slow down.");
}
```

■ ***Tip:*** *synchronized on tryAcquire() makes refill + check + decrement atomic. For distributed multi-node systems: swap AtomicLong for Redis Lua script (see Q13). Same algorithm, different scope.*

■ SECTION 7 · QUICK REFERENCE CHEATSHEET

Kafka Production Config — Copy-Paste Ready

Property	Recommended Value	Why
acks	all	All ISR must ack — zero message loss
enable.idempotence	true	Broker deduplicates retried producer messages
isolation.level	read_committed	Skip messages from aborted transactions
auto.offset.reset	earliest	Don't miss messages on new consumer group
enable.auto.commit	false	Manual commit = full control over offset
partition.assignment.strategy	CooperativeStickyAssignor	Zero-downtime incremental rebalances
session.timeout.ms	45000	Allows slow processing consumers to breathe
max.poll.interval.ms	300000	5 min — prevent false rebalance on heavy DB calls
max.poll.records	100	Don't over-fetch; avoid poll interval violations

@Transactional Propagation — Quick Reference

Propagation	Behavior	Use When
REQUIRED (default)	Join existing tx or create new	Standard — most service methods
REQUIRES_NEW	Suspend outer, always new tx	Audit logs, outbox events — must save even if outer fails
NESTED	Savepoint within outer tx	Partial rollback without failing entire outer tx
MANDATORY	Throw if no existing tx	Assert code must run inside a transaction
NEVER	Throw if tx exists	Read-only cache operations that must not be in a tx

Performance Numbers — Know These by Heart

Component	Benchmark	Notes for Paytm
Redis GET/SET	~100K ops/sec, < 1ms	Rate limiting, idempotency, balance cache, session

MySQL InnoDB	~10K TPS, ~5ms per query	HikariCP pool = (cores×2) + spindles
Kafka throughput	1M+ msgs/sec per broker	Tune batch.size=64KB, linger.ms=5
JVM G1GC pause	100–200ms	Use ZGC (< 1ms) for payment API SLA
HTTP internal (same VPC)	1–5ms	Service-to-service on same data center
HTTP external API	50–200ms	Bank gateway, UPI NPCI, card network
HikariCP default pool	10 connections	Always increase! Payment service: 50–100
CompletableFuture gain	60–70% latency reduction	Parallelize 3×300ms serial calls → 300ms total

Best of Luck with Your Paytm Interview! ■

Focus order: Kafka (highest weight) → System Design → Spring Boot internals → Behavioral STAR